

Corrected Lab Manual

pgBackRest Backup, Restore and PITR

PostgreSQL 18 on Kubernetes with Patroni / Spilo

Environment used in this lab

Item	Value
Namespace	postgres-ha
Primary pod	acid-postgres-ha-0
Replica pods	acid-postgres-ha-1, acid-postgres-ha-2
PostgreSQL	18.3
pgBackRest	2.59.1
Confirmed PGDATA	/home/postgres/pgdata/pgroot/data
Lab config	/home/postgres/pgdata/pgbackrest.conf
Lab repository	/home/postgres/pgdata/pgbackrest-repo

Purpose: This corrected edition incorporates the command errors observed during the lab: missing `kubectl exec -- separator`, root execution of `pgBackRest`, wrong ownership, `chmod/chown` confusion, and mistyped `--stanza`.

Copy commands one line at a time. Do not type prompt characters such as `rps@ubuntu24:~$`.

1. Lab safety and command conventions

Important: This repository location is suitable for a classroom lab because it is placed on the PostgreSQL pod persistent volume. It is not an HA repository design. After failover, another pod may not see the same repository. Production designs normally use shared/object storage such as S3, NFS, or an RWX PVC.

All commands in this manual are run from the Ubuntu terminal unless a step explicitly says otherwise. The Kubernetes pod command separator `--` is mandatory with your kubectl version.

```
kubectl exec -n <namespace> <pod> -- <command>
```

Use `chown` to change ownership and `chmod` to change permissions:

```
chown postgres:postgres /path/to/file
chmod 640 /path/to/file
```

pgBackRest stanza and backup commands must run as the postgres operating-system user in this container. Changing file ownership alone does not change the user that executes the command.

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- su -s /bin/sh postgres -c 'pgbackrest ...'
```

2. Pre-check: identify leader and replicas

Check the database pods and Spilo role labels:

```
kubectl get pods -n postgres-ha -L spilo-role
```

Expected database roles in the current lab:

Pod	Role
acid-postgres-ha-0	master / leader / primary
acid-postgres-ha-1	replica
acid-postgres-ha-2	replica

Do not use pooler pods: Pods named `acid-postgres-ha-pooler-*` are PgBouncer connection poolers, not PostgreSQL database members. Do not select them as the leader.

Verify the primary directly from PostgreSQL:

```
kubectl exec -n postgres-ha acid-postgres-ha-0 -- psql -U postgres -d postgres -Atc "SELECT CASE WHEN
pg_is_in_recovery() THEN 'REPLICA' ELSE 'PRIMARY' END;"
```

Expected output: PRIMARY

3. Set and verify lab variables

```
export NS=postgres-ha
export PRIMARY=acid-postgres-ha-0
export STANZA=acid-postgres-ha
export PGDATA=/home/postgres/pgdata/pgroot/data
export PGBR_CFG=/home/postgres/pgdata/pgbackrest.conf
export PGBR_REPO=/home/postgres/pgdata/pgbackrest-repo
```

Verify before continuing:

```
echo $NS
echo $PRIMARY
echo $STANZA
echo $PGDATA
echo $PGBR_CFG
echo $PGBR_REPO
```

4. Confirm PGDATA and pgBackRest

Confirm the container environment variable:

```
kubectl exec -n $NS $PRIMARY -- printenv PGDATA
```

Expected:

```
/home/postgres/pgdata/pgroot/data
```

Confirm from PostgreSQL itself; this is the authoritative value for pg1-path:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -Atc "SHOW data_directory;"
```

Check pgBackRest version:

```
kubectl exec -n $NS $PRIMARY -- pgbackrest version
```

Expected in this lab: pgBackRest 2.59.1

5. Verify persistent storage

Check the PostgreSQL persistent filesystem:

```
kubectl exec -n $NS $PRIMARY -- df -hT /home/postgres/pgdata
```

Also inspect the mount:

```
kubectl exec -n $NS $PRIMARY -- mount | grep /home/postgres/pgdata
```

Why this check matters: Earlier, /var/lib/pgbackrest resolved to the container overlay filesystem. A backup repository on overlay can disappear with the pod. This lab therefore uses /home/postgres/pgdata/pgbackrest-repo.

6. Create the repository directory

```
kubectl exec -n $NS $PRIMARY -- mkdir -p $PGBR_REPO
kubectl exec -n $NS $PRIMARY -- chown -R postgres:postgres $PGBR_REPO
```

Verify ownership:

```
kubectl exec -n $NS $PRIMARY -- ls -ld $PGBR_REPO
```

Expected owner/group: postgres postgres

Test write access as postgres:

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'touch /home/postgres/pgdata/pgbackrest-repo/.write-test && echo WRITE_OK && rm /home/postgres/pgdata/pgbackrest-repo/.write-test'
```

Expected: WRITE_OK

7. Create the pgBackRest configuration inside the pod

Do not create /etc/pgbackrest/pgbackrest.conf from the Ubuntu host: The earlier No such file or directory error occurred because `cat > /etc/pgbackrest/pgbackrest.conf` was executed on ubuntu24, not inside the PostgreSQL pod.

Create the file by piping it into the pod:

```
kubectl exec -i -n $NS $PRIMARY -- sh -c 'cat > /home/postgres/pgdata/pgbackrest.conf' <<'EOF'
[global]
repo1-path=/home/postgres/pgdata/pgbackrest-repo
repo1-retention-full=2
repo1-retention-diff=3
start-fast=y
process-max=2
log-level-console=info

[acid-postgres-ha]
pg1-path=/home/postgres/pgdata/pgroot/data
EOF
```

Because the file was created through `kubectl exec` as the container default user, correct its ownership and permissions:

```
kubectl exec -n $NS $PRIMARY -- chown postgres:postgres $PGBR_CFG
kubectl exec -n $NS $PRIMARY -- chmod 640 $PGBR_CFG
```

Verify:

```
kubectl exec -n $NS $PRIMARY -- ls -l $PGBR_CFG
kubectl exec -n $NS $PRIMARY -- cat $PGBR_CFG
```

Expected ownership: postgres postgres. Expected mode: -rw-r----

8. Confirm which user kubectl exec uses

This explains the root error seen during stanza-create:

```
kubectl exec -n $NS $PRIMARY -- id
```

If the result is `uid=0(root)`, a direct pgbackrest command runs as root and stanza-create is rejected.

Confirm the postgres OS account:

```
kubectl exec -n $NS $PRIMARY -- id postgres
```

Your lab showed `uid=101(postgres)`, `gid=103(postgres)`.

Test pgBackRest as postgres:

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest version'
```

9. Create the pgBackRest stanza

Run stanza-create as the postgres OS user:

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest
--config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha stanza-create'
```

Expected ending:

```
INFO: stanza-create command end: completed successfully
```

Typing check: The option must be exactly `--stanza=acid-postgres-ha`. Do not use a corrupted string such as `--^Canza` or `--stanza` with missing characters.

Display stanza information:

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest
--config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha info'
```

10. Inspect existing WAL archive settings before changing them

Patroni/Spilo may already manage WAL archiving. Inspect first:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SHOW archive_mode;"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SHOW archive_command;"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT archived_count, failed_count,
last_archived_wal, last_failed_wal FROM pg_stat_archiver;"
```

Do not overwrite production archiving blindly: If `archive_command` already uses WAL-G, WAL-E, or another production backup system, do not replace it. Use a disposable training cluster or configure pgBackRest through your Patroni/Spilo deployment settings.

11. Configure `archive_command` for this disposable lab

Use this step only when `archive_mode` is already on and replacing `archive_command` is acceptable for the lab.

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "ALTER SYSTEM SET archive_command =
'pgbackrest --config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha archive-push
%p';"
```

Reload configuration:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT pg_reload_conf();"
```

Verify:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SHOW archive_command;"
```

Patroni note: Patroni can re-apply managed parameters. If `archive_command` later reverts, configure the setting through Patroni/Spilo rather than repeatedly using `ALTER SYSTEM`.

12. Force WAL and run pgBackRest check

Force a WAL switch:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT pg_switch_wal();"
```

Wait a few seconds, then inspect the archiver:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT archived_count, failed_count,
last_archived_wal, last_failed_wal FROM pg_stat_archiver;"
```

Then run the pgBackRest check as postgres:

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest
--config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha check'
```

Expected ending: check command end: completed successfully

13. Prepare sample data

Create a table used throughout the backup and PITR exercises:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "CREATE TABLE IF NOT EXISTS
pgbackrest_lab (id integer PRIMARY KEY, description text, created_at timestamptz DEFAULT
clock_timestamp());"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "INSERT INTO
pgbackrest_lab(id,description) VALUES (1,'Before full backup'),(2,'Full backup baseline') ON CONFLICT
(id) DO NOTHING;"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT * FROM pgbackrest_lab ORDER
BY id;"
```

14. Lab Exercise 1 - Full backup

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest
--config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha --type=full backup'
```

Review the backup catalog:

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest
--config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha info'
```

Expected: at least one backup with type full.

15. Lab Exercise 2 - Incremental backup

Create changes after the full backup:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "INSERT INTO
pgbackrest_lab(id,description) VALUES (3,'Created after full backup'),(4,'Included in incremental
backup') ON CONFLICT (id) DO NOTHING;"
```

Run the incremental backup:

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest
--config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha --type=incr backup'
```

Verify with info:

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest
--config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha info'
```

Concept: An incremental backup contains changes since the most recent successful backup in its chain.

16. Lab Exercise 3 - Differential backup

Create more changes:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "INSERT INTO
pgbackrest_lab(id,description) VALUES (5,'Created before differential backup'),(6,'Differential
backup test') ON CONFLICT (id) DO NOTHING;"
```

Run the differential backup:

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest
--config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha --type=diff backup'
```

Verify:

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest
--config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha info'
```

Concept: A differential backup contains changes since the most recent full backup.

17. Lab Exercise 4 - Safe restore to an alternate directory

Safety: Do not restore directly over the live Patroni leader PGDATA while Patroni/PostgreSQL is running. This lab restores to a separate directory for inspection.

Create an empty restore directory and give it to postgres:

```
kubectl exec -n $NS $PRIMARY -- sh -c 'rm -rf /home/postgres/pgdata/restore-test && mkdir -p
/home/postgres/pgdata/restore-test && chown postgres:postgres /home/postgres/pgdata/restore-test'
```

Restore the latest backup set into the alternate directory:

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest
--config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha
--pg1-path=/home/postgres/pgdata/restore-test restore'
```

Inspect the restored cluster files:

```
kubectl exec -n $NS $PRIMARY -- ls -lah /home/postgres/pgdata/restore-test | head -30
kubectl exec -n $NS $PRIMARY -- cat /home/postgres/pgdata/restore-test/PG_VERSION
```

Expected PG_VERSION: 18

18. Lab Exercise 5 - Point-in-Time Recovery (PITR)

PITR uses a base backup plus archived WAL to stop recovery at a selected time.

Step A - insert a row that must exist at the recovery target:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "INSERT INTO
pgbackrest_lab(id,description) VALUES (100,'KEEP THIS ROW FOR PITR') ON CONFLICT (id) DO UPDATE SET
description=EXCLUDED.description;"
```

Step B - record a target timestamp immediately after the insert:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -Atc "SELECT clock_timestamp();"

```

Write down the timestamp: Example only: 2026-09-25 02:15:30.123456+00. Use your actual output in the restore command.

Step C - wait a few seconds and simulate accidental deletion:

```
sleep 5
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "DELETE FROM pgbackrest_lab WHERE
id=100;"
```

Verify the row is gone:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT * FROM pgbackrest_lab WHERE
id=100;"
```

Step D - force WAL archiving and verify no new archive failure:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT pg_switch_wal();"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "SELECT archived_count, failed_count,
last_archived_wal, last_failed_wal FROM pg_stat_archiver;"
```

Step E - create an empty PITR restore directory owned by postgres:

```
kubectl exec -n $NS $PRIMARY -- sh -c 'rm -rf /home/postgres/pgdata/pitr-test && mkdir -p
/home/postgres/pgdata/pitr-test && chown postgres:postgres /home/postgres/pgdata/pitr-test'
```

Step F - restore to your recorded time. Replace the example timestamp:

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest
--config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha
--pg1-path=/home/postgres/pgdata/pitr-test --type=time --target="2026-09-25 02:15:30.123456+00" --
target-action=promote restore'
```

Inspect the restored directory:

```
kubectl exec -n $NS $PRIMARY -- ls -lah /home/postgres/pgdata/pitr-test | head -30
```

Validation scope: The restore command prepares an independent recovered PGDATA. Starting that recovered cluster should be done in an isolated recovery instance or carefully on a separate port; do not point Patroni at it while the live cluster is running.

19. Quick verification commands

What to check	Command / result
Leader/replicas	<code>kubectl get pods -n postgres-ha -L spilo-role</code>
Primary check	<code>pg_is_in_recovery() = f</code> on primary
PGDATA	<code>/home/postgres/pgdata/pgroot/data</code>
Config owner	<code>postgres postgres</code>
Repository write	<code>WRITE_OK</code>
Stanza	<code>stanza-create completed successfully</code>
WAL archive	<code>failed_count</code> should not increase
Backup catalog	<code>pgbackrest ... info</code>
Full backup	<code>type=full</code>
Incremental	<code>type=incr</code>
Differential	<code>type=diff</code>
Restore	<code>PG_VERSION = 18</code> in <code>restore-test</code>

20. Troubleshooting - errors seen in this lab

Error / symptom	Cause	Correct action
exec [POD] [COMMAND] is not supported anymore	Missing -- between pod and command	Use: kubectl exec -n \$NS \$PRIMARY -- command
id: pstgres: no such user	Typo in postgres username	Use: id postgres
stanza-create must not be run as root	kubectl exec default user is root	Run pgBackRest through su -s /bin/sh postgres -c
root root on pgbackrest.conf	File was created as root	chown postgres:postgres then chmod 640
chmod 640 postgres:postgres ...	chmod and chown were combined incorrectly	chown owner first; chmod mode second
No such file /etc/pgbackrest/...	File was created on Ubuntu host instead of in pod	Use kubectl exec -i ... cat > /home/postgres/pgdata/pgbackrest.conf
touch: missing file operand	Command was split at the wrong place	Paste the full test command on one line
--^Canza or invalid option	Control character / typing error	Type exactly --stanza=acid-postgres-ha
Repository on overlay	Path is not on persistent storage	Use the lab repository under /home/postgres/pgdata

21. Copy-safe command sequence (current lab state)

If your leader is still acid-postgres-ha-0, this is the shortest corrected setup sequence. Run one line at a time:

```
export NS=postgres-ha
export PRIMARY=acid-postgres-ha-0
export STANZA=acid-postgres-ha
export PGBR_CFG=/home/postgres/pgdata/pgbackrest.conf
export PGBR_REPO=/home/postgres/pgdata/pgbackrest-repo
kubectl exec -n $NS $PRIMARY -- chown postgres:postgres $PGBR_CFG
kubectl exec -n $NS $PRIMARY -- chmod 640 $PGBR_CFG
kubectl exec -n $NS $PRIMARY -- mkdir -p $PGBR_REPO
kubectl exec -n $NS $PRIMARY -- chown -R postgres:postgres $PGBR_REPO
kubectl exec -n $NS $PRIMARY -- ls -l $PGBR_CFG
kubectl exec -n $NS $PRIMARY -- ls -ld $PGBR_REPO
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'touch /home/postgres/pgdata/pgbackrest-repo/.write-test && echo WRITE_OK && rm /home/postgres/pgdata/pgbackrest-repo/.write-test'
```

```
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest version'
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest
--config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha stanza-create'
kubectl exec -n $NS $PRIMARY -- su -s /bin/sh postgres -c 'pgbackrest
--config=/home/postgres/pgdata/pgbackrest.conf --stanza=acid-postgres-ha info'
```

22. Production design note

This lab deliberately keeps the repository close to the database to make the exercises easy to reproduce. For an HA production system, design backups independently of a single database pod or node.

Lab design	Production direction
Repository under one PostgreSQL PVC	Shared/object repository reachable from every eligible backup/restore host
Commands executed manually	Automation/operator integration with least privilege
archive_command changed manually	Patroni/Spilo managed configuration
Restore to alternate folder for practice	Documented cluster-wide recovery runbook and replica rebuild procedure
Single local copy	Retention, encryption, off-site copy, monitoring and restore testing

23. Lab completion checklist

- ☐ Leader and replicas identified correctly.
- ☐ PGDATA confirmed as /home/postgres/pgdata/pgroot/data.
- ☐ Repository is on persistent lab storage and writable by postgres.
- ☐ pgbackrest.conf is owned by postgres:postgres and mode 640.
- ☐ stanza-create runs as postgres, not root.
- ☐ WAL archive settings were inspected before modification.
- ☐ pgBackRest check completes successfully.
- ☐ Full backup completes and appears in info.
- ☐ Incremental backup completes and appears in info.
- ☐ Differential backup completes and appears in info.
- ☐ Restore to restore-test completes without touching the live PGDATA.
- ☐ PITR restore is performed to pitr-test using a recorded target time.

End of lab: Keep the repository and restored test directories only as long as needed for the exercise. In a disposable lab you may remove restore-test and pitr-test after validation.